Parallel Quicksort with OpenMPI, Step by Step

From an empty folder to hypercube quicksort (Algorithm 9.9), a pool of tasks, cluster runs on Apini and Speed, and the written questions

COMP 428 / 6281: Parallel Programming — Assignment 2

Gina Cody School of Engineering and Computer Science
Concordia University

Fall 2026

Reference: A. Grama, A. Gupta, G. Karypis, V. Kumar, *Introduction to Parallel Computing*, 2nd ed., 2003 — §3.6.3, §9.4.3–9.4.4, Algorithm 9.9, Figure 9.21, Problems 9.17–9.21, 9.24.
Every code listing in this deck is read directly from `quicksort_mpi/tutorial/`, which is built and tested.

What We Will Build

The assignment (programming part)

1. Sequential quicksort on one node, timed over a broad range of input lengths.
2. Parallel quicksort on a *virtual hypercube* (Problem 9.17, Algorithm 9.9), workers on **distinct nodes**.
3. Run times for $p \in \{1, 2, 4, 6, 8\}$ on **Apini** (5 nodes $\times$ 2 cores: at most 10 workers) and $p \in \{12, 16\}$ on **Speed**.
4. Plot speedup vs. workers and vs. input length; explain.

Submit: the sequential version, the statically assigned parallel version and the **pool-of-tasks** parallel version.

Written part covered here

Q1: T_P on a physical hypercube. Q2: the same algorithm on a $\sqrt{p} \times \sqrt{p}$ mesh.

The steps of this tutorial

0	Project, Makefile, "hello" on the cluster
1	Sequential quicksort (Req. 1)
–	*The textbook algorithm*
2	Distribute the input
3	One split: pivot + exchange
4	All d splits, collect, verify (Req. 2)
5	p that is not a power of two
6	Pool of tasks
7	Testing
8	Running on Apini and Speed
9	Measuring and plotting (Req. 3, 4)
–	*Written questions 1 and 2*

Each step ends with a **checkpoint**: a command whose output tells you the step works.

Step 0: The Project and Its Makefile

The folder grows one file per step:

```
tutorial/
  Makefile
  step0_hello.cpp       Step 0
  quicksort.hpp         Step 1
  step1_sequential.cpp  Step 1
  step2_hypercube.cpp   Steps 2-5
  step3_pool.cpp        Step 6
  test_quicksort.cpp    Step 7
  test_mpi.sh           Step 7
  bench.sh              Step 9
  plot_speedup.py       Step 9
```

- One rule builds every `stepN_*.cpp` into `bin/stepN_*`: add a file to `PROGS` and you are done.
- Use `mpicxx` for everything, even the sequential program. Then there is only one compiler and one set of flags.
- No `-march=native`: Apini/Speed nodes may have different CPUs.

```
# Tutorial build (the simple version the slides walk through).
#   make            build bin/step0_hello ... bin/step3_pool
#   make test       unit test + verified MPI runs on this machine
MPICXX   ?= mpicxx
CXXFLAGS ?= -O2 -std=c++17 -Wall -Wextra
MPIRUN   ?= mpirun
MPIFLAGS ?= --oversubscribe

PROGS := bin/step0_hello bin/step1_sequential bin/step2_hypercube bin/step3_pool

.PHONY: all test clean
all: $(PROGS)

bin/%: %.cpp quicksort.hpp | bin
  $(MPICXX) $(CXXFLAGS) $< -o $@

bin/test_quicksort: test_quicksort.cpp quicksort.hpp | bin
  $(MPICXX) $(CXXFLAGS) $< -o $@

test: $(PROGS) bin/test_quicksort
  ./bin/test_quicksort
  MPIRUN="$(MPIRUN)" MPIFLAGS="$(MPIFLAGS)" bash test_mpi.sh

bin:
  mkdir -p $@

clean:
  rm -rf bin
```

Step 0: Hello, Cluster

```cpp
// step0_hello.cpp -- Step 0: check the MPI toolchain and the process mapping.
// Every rank prints the node it runs on; with the hostfile, all names differ.
#include <mpi.h>
#include <cstdio>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size, len;
    char host[MPI_MAX_PROCESSOR_NAME];
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(host, &len);
    std::printf("rank %d of %d on %s\n", rank, size, host);
    MPI_Finalize();
    return 0;
}
```

```
make
# laptop: 4 ranks on 1 machine
mpirun --oversubscribe -np 4 ./bin/step0_hello

# Apini: 4 ranks on 4 different nodes
mpirun -np 4 --map-by node \
  --mca btl_tcp_if_include ens18 \
  --mca oob_tcp_if_include ens18 \
  --hostfile /media/pkg/apini-scripts/apini_hostfile \
  ./bin/step0_hello
```

Why start here

It tests the compiler, `mpirun`, the hostfile and the mapping *before* there is any sorting code to suspect. Requirement 2 says "distinct nodes", and this program shows them.

Why `--map-by node`

The Apini hostfile lists 5 nodes with `slots=2` (2 cores each). By default OpenMPI *fills a node first*, so ranks 0 and 1 share a machine. `--map-by node` deals ranks round-robin: $p \leq 5$ get distinct nodes; $p = 6, 8$ must share some.

Checkpoint

Four lines `rank r of 4 on <host>` with **four different host names**.

Step 1: The Sequential Kernel (`quicksort.hpp`)

```
10 // Hoare partition of a[lo..hi] around a randomly chosen pivot.
11 // Returns j such that every element of a[lo..j] is <= every element of
12 // a[j+1..hi], with lo <= j < hi (both sides are non-empty).
13 inline long partition(int* a, long lo, long hi) {
14     static std::mt19937 rng(12345);
15     // A random pivot makes no input special: sorted or reversed input is
16     // O(n log n) on average, like any other.
17     std::swap(a[lo], a[lo + static_cast<long>(rng() % (hi - lo + 1))]);
18     const int pivot = a[lo];
19     long i = lo - 1, j = hi + 1;
20     while (true) {
21         do { ++i; } while (a[i] < pivot);
22         do { --j; } while (a[j] > pivot);
23         if (i >= j) return j;
24         std::swap(a[i], a[j]);
25     }
26 }
27
28 // Sorts a[lo..hi]. Recursing only into the smaller side and looping over the
29 // larger one keeps the stack depth O(log n) whatever the pivots are.
30 inline void quicksort(int* a, long lo, long hi) {
31     while (lo < hi) {
32         const long j = partition(a, lo, hi);
33         if (j - lo < hi - j) {
34             quicksort(a, lo, j);
35             lo = j + 1;
36         } else {
37             quicksort(a, j + 1, hi);
38             hi = j;
39         }
40     }
41 }
```

- **Hoare partition** (lines 19–25): two indices move towards each other and swap misplaced pairs. It handles many equal keys well: all-equal input splits in the middle.
- **Random pivot**, swapped to `a[lo]` first (line 17). With the pivot at `a[lo]`, Hoare's loop always returns $j < hi$, so both sides are non-empty and the recursion terminates.
- **Why random?** Every input is then "average". A fixed rule such as median-of-three has inputs that make it quadratic, and a parallel algorithm produces exactly such structured inputs ("A bug worth studying", near the end).
- **Smaller side first** (lines 33–39): the recursion depth is at most $\log_2 n$ whatever the pivots are, so there is no stack overflow at $n = 5 \cdot 10^7$.

Step 1: Driver, Timing and Checking (Requirement 1)

```cpp
int main(int argc, char** argv) {
    const long n = argc > 1 ? std::atol(argv[1]) : 1000000;
    const unsigned seed = argc > 2 ? std::atoi(argv[2]) : 1;

    std::vector<int> a = random_input(n, seed);
    std::vector<int> expected = a;  // for the check only, not timed
    std::sort(expected.begin(), expected.end());

    const auto t0 = std::chrono::steady_clock::now();
    quicksort(a);
    const auto t1 = std::chrono::steady_clock::now();

    const double seconds = std::chrono::duration<double>(t1 - t0).count();
    const bool ok = (a == expected);
    std::printf("sequential,1,1,%ld,%.6f,%d\n", n, seconds, ok ? 1 : 0);
    return ok ? 0 : 1;
}
```

- `random_input(n, seed)` (in `quicksort.hpp`) gives every program the *same* input for the same seed, so the versions can be compared.
- Only the sort is timed. The check against `std::sort` runs outside the timer.
- Every program prints **one CSV row**: `version,p,active,n,seconds,ok`. Step 9 pastes the rows together.

```
./bin/step1_sequential 1000000
sequential,1,1,1000000,<seconds>,1

# Requirement 1 on one cluster node:
H=/media/pkg/apini-scripts/apini_hostfile
for n in 100000 1000000 10000000; do
  mpirun -np 1 --hostfile $H \
    ./bin/step1_sequential $n
done
```

Checkpoint

`ok = 1` for every n, and the time grows a little faster than linearly ($n \log n$): $10\times$ more elements takes about 11–12$\times$ longer.

Note

`mpirun -np 1` also runs non-MPI programs. It is the easy way to put the sequential run on a cluster node instead of the login node.

Problem 9.17: The Setting

- n elements, $p = 2^d$ processes in a d-dimensional hypercube. Each process starts with a block of n/p elements.
- **The process labels define the global order**: at the end, every element on process r must be $\leq$ every element on process $r + 1$.
- A pivot is selected and **broadcast**. Each process splits its block into "smaller" and "larger" parts.
- Processes connected along the d-th link (the **most significant bit**) exchange parts:
  - label bit $= 0 \Rightarrow$ keeps the *smaller* elements;
  - label bit $= 1 \Rightarrow$ keeps the *larger* elements.
- Now the $(d-1)$-subcube `0**...` holds only elements $\leq$ pivot and `1**...` only elements $>$ pivot. **Recurse in each subcube**, one dimension per step.
- After d splits the sequence is sorted *with respect to the labels*, but not inside each process. So each process finishes with a **sequential quicksort** (our Step 1).

Key difference from sequential quicksort

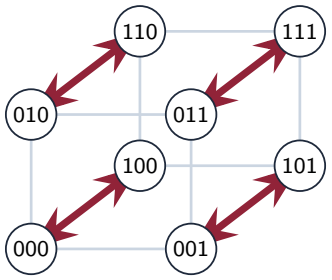
The recursion is over *process groups*, not over array ranges. The group size halves every step regardless of the pivot. A bad pivot therefore leaves processes with too much or too little data **permanently** (§9.4.4).

Algorithm 9.9, Line by Line

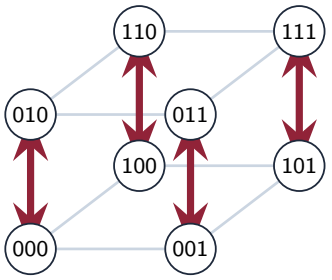
```
1  procedure HYPERCUBE_QUICKSORT (B, n)
2  begin
3     id := process's label;
4     for i := 1 to d do
5     begin
6        x := pivot;
7        partition B into B1 and B2 such that B1 ≤ x < B2;
8        if i^th bit is 0 then
9        begin
10          send B2 to the process along the i^th communication link;
11          C := subsequence received along the i^th communication link;
12          B := B1 ∪ C;
13       endif
14       else
15          send B1 to the process along the i^th communication link;
16          C := subsequence received along the i^th communication link;
17          B := B2 ∪ C;
18       endelse
19    endfor
20    sort B using sequential quicksort;
21 end HYPERCUBE_QUICKSORT
```

- **Line 6 — pivot.** "$x := pivot$" hides the hard part: every process of the *current subcube* must agree on x. So this is a collective inside the subcube (§9.4.4).
- **Line 7 — local split**, $\Theta(|B|)$, no communication. Note the tie rule: $B_1 \leq x < B_2$.
- **Lines 8–18 — exchange.** "The ith bit" with $i = 1 \ldots d$ means the dimensions are taken from the most significant bit down (Fig. 9.21 splits the 3rd dimension first). We use bit $k = d - i$ and partner $= id \oplus 2^k$.
- Both partners send *and* receive, and $|C|$ is not known in advance.
- **Line 20 — local sort**: the bulk of the work when pivots are good.
- Not in the pseudocode: getting the input *onto* the processes and the result *off* them. We need both (Steps 2 and 4).

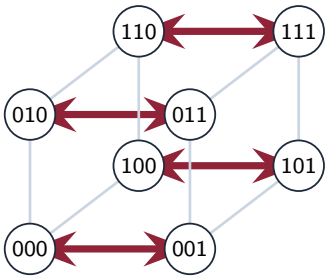
Figure 9.21 Revisited: The Three Splits for $d = 3$



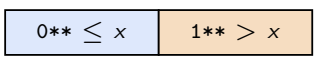
(a) split along bit 2 (MSB)

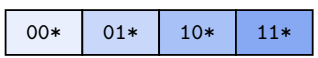


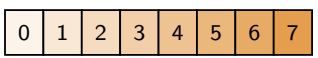
(b) split along bit 1



(c) split along bit 0







Step i	Bit $k = d - i$	Subcubes choosing a pivot	Exchanging pairs
1	2	one: *** (8 processes)	$(0,4),(1,5),(2,6),(3,7)$
2	1	two: 0**, 1** (4 each)	$(0,2),(1,3)$ and $(4,6),(5,7)$
3	0	four: 00*, 01*, 10*, 11*	$(0,1),(2,3),(4,5),(6,7)$

Bottom row: the order the data ends up in (textbook Fig. 9.21, middle column). After step 3 the blocks are ordered by process label.

Why It Sorts (Problem 9.18)

Claim. After step i, for any two subcubes Q, Q' of dimension $d - i$ (processes sharing their top i label bits), if $\text{label}(Q) < \text{label}(Q')$ then every element in Q is $\leq$ every element in Q'.

Proof by induction on i

- $i = 0$**:** one subcube, nothing to show.
- **Step** i**:** take a $(d - i + 1)$-subcube R from step $i - 1$. All its processes use the same pivot x. Each pair $(u, u \oplus 2^k)$ inside R exchanges. Afterwards the half of R with bit $k = 0$ holds exactly the elements of R that are $\leq x$, and the half with bit $k = 1$ holds exactly those $> x$. So the two new subcubes are ordered.
- Elements never leave R: partners differ only in bit k, which lies below R's fixed prefix. So the order between different R's from step $i - 1$ is preserved.
- After $i = d$ every subcube is a single process, so the processes are ordered by label. A local sort on each process then gives a globally sorted sequence. $\square$

What this tells the implementer

- Correctness needs **only** that all members of a subcube agree on x. *Any* pivot is correct; pivot quality only affects speed (load balance).
- The pivot collective must be confined to the current subcube: we need one communicator per subcube (Step 4).

From Pseudocode to MPI: The Plan for Steps 2–5

Algorithm 9.9	MPI	Where
(input on every process)	`MPI_Scatterv` from rank 0	Step 2: `scatter_blocks`
line 3: `id`	`MPI_Comm_rank(cube)`	`main`
line 6: $x :=$ pivot	`MPI_Allgather` of local medians in the subcube	Step 3: `choose_pivot`
line 7: partition	`std::partition(v <= x)`	Step 3: `split`
lines 8–18: exchange	partner $= id \oplus 2^k$, two `MPI_Sendrecv` (size, then data)	Step 3: `split`
line 4: **for** $i := 1$ **to** d	`for (k = d-1; k >= 0; --k)`	Step 4: `main`
"recurse in each subcube"	`MPI_Comm_split(sub, bit k)`	Step 4: `main`
line 20: local sort	`quicksort(B)` from Step 1	Step 4: `main`
(result on one process)	`MPI_Gather` sizes + `MPI_Gatherv`	Step 4: `gather_blocks`
$p = 2^d$	fold p to $2^{\lfloor \log_2 p \rfloor}$ active ranks	Step 5: `main`

All of it goes into one file, `step2_hypercube.cpp`, built up function by function on the next slides.

Step 2: Distribute the Input

```cpp
// ---- Step 2: distribute the input in blocks of ~n/p -----------------------
std::vector<int> scatter_blocks(const std::vector<int>& A, long n, MPI_Comm comm) {
    int rank, p;
    MPI_Comm_rank(comm, &rank);
    MPI_Comm_size(comm, &p);
    std::vector<int> counts(p), displs(p);
    for (int r = 0; r < p; ++r) {
        counts[r] = static_cast<int>(n / p + (r < n % p ? 1 : 0));
        displs[r] = r == 0 ? 0 : displs[r - 1] + counts[r - 1];
    }
    std::vector<int> B(counts[rank]);
    MPI_Scatterv(A.data(), counts.data(), displs.data(), MPI_INT,
                 B.data(), counts[rank], MPI_INT, 0, comm);
    return B;
}
```

- Rank 0 generates the input (it is the only one with a non-empty `A`).
- n need not be divisible by p: the first $n \bmod p$ ranks get one extra element. `counts` and `displs` describe which slice of `A` goes where; every rank computes them identically.
- `MPI_Scatterv` is a collective: *all* ranks of `comm` call it. Only the root reads the send arguments.
- Elements are `int`, so `MPI_INT`. Counts are `int` too, which is fine up to $2^{31} - 1$ elements per message.

Checkpoint

Temporarily print `rank` and `B.size()`: for $n = 10$, $p = 4$ you get $3, 3, 2, 2$.